

Índice

1 El problema	1
2 Implementación	2
2.1 Formato de la entrada	2
2.2 Formato de la salida	2
3 Archivos de partida	2
3.1 Archivos de prueba: public.zip	3
3.2 Archivo principal: e5.py	3
4 Entrega en el aula virtual	4
5 Calificación del entregable	5
5.1 Medición de tiempos de ejecución	5
5.2 Errores graves en el entregable	6
5.3 Penalización por copia	6

1. El problema

Un puzle deslizante de 3×3 consiste en un tablero de 3×3 casillas que contiene ocho fichas numeradas del 1 al 8 y un hueco vacío. Para resolver el puzle debemos colocar las fichas en orden creciente dejando el hueco en la última fila y columna. En cada paso podemos mover una ficha adyacente al hueco, que intercambia su posición con el hueco. La figura 1 muestra un ejemplo.

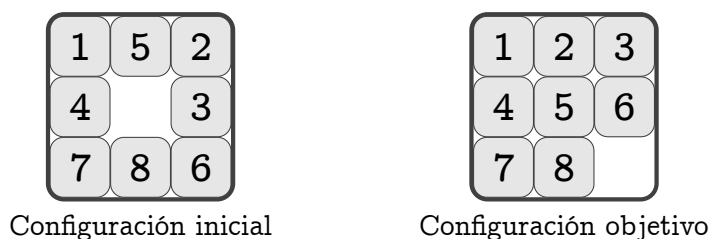


Figura 1: Configuración inicial y final de un puzle deslizante de 3×3 .

Queremos encontrar la secuencia más corta de movimientos que resuelva el puzle. En este tipo de puzles es habitual indicar los movimientos desde el punto de vista del hueco, que puede moverse en cuatro direcciones (arriba, abajo, izquierda, derecha), siempre que el movimiento no lo saque del tablero. En cada paso, la ficha que ocupa el destino del hueco se desliza hacia la posición vacía. Para el puzle de la figura 1 la solución más corta es arriba, derecha, abajo, abajo.

2. Implementación

Implementad un programa de línea de órdenes, `e5.py`, que reciba por la entrada estándar el puzle en el formato que se especifica en el apartado 2.1. Como resultado de su ejecución el programa mostrará por la salida estándar la secuencia de pasos más corta que lo resuelve, en el formato que se especifica en el apartado 2.2.

Obligatoriamente, vuestra implementación debe resolver el problema utilizando explícitamente el esquema de ramificación y acotación de la biblioteca `algoritmia` (`bab_scheme.py`).

Se garantiza que todas las instancias tendrán solución.

2.1. Formato de la entrada

Una instancia es un archivo de texto cuyo contenido es un puzle representado con tres líneas de texto, cada una con tres enteros separados por un espacio en blanco. El hueco se representa mediante el número 0.

Como ejemplo, veamos el contenido del archivo `e5-01-4.i`:

```
1 5 2
4 0 3
7 8 6
```

2.2. Formato de la salida

El programa deberá mostrar una línea de texto por la salida estándar con la secuencia de movimientos más corta que resuelve el puzle. Utilizaremos los caracteres R (*Right*), L (*Left*), U (*Up*) y D (*Down*) para representar el movimiento del hueco.

Por ejemplo, el archivo `e5-01-4.o` contiene una línea de texto con la solución de la instancia `e5-01-4.i`:

```
URDD
```

Es posible que existan varias secuencias de movimiento de longitud mínima. Todas se considerarán válidas en la calificación del entregable.

3. Archivos de partida

En el aula virtual disponéis de una carpeta con el siguiente contenido:

- `public.zip`: un archivo comprimido que contiene la carpeta `public` con un conjunto de diez instancias de prueba y sus resultados.
- `e5.py`: el programa que debéis utilizar como punto de partida y que hay que entregar.

En los siguientes subapartados se explican con detalle estos contenidos, pero ya podéis crear un nuevo proyecto PyCharm y añadirle el archivo `e5.py` y la carpeta que contiene el archivo `public.zip`.

Por otro lado, en <https://rei.dlsi.uji.es/algoritmia/viewer3x3.html> disponéis de un visor donde podéis introducir un puzle y una secuencia de movimientos y ver una animación paso a paso. Para conectaros a esta página desde fuera de la UJI debéis utilizar la VPN de la universidad, tal y como se explica en el apartado 5.1.

3.1. Archivos de prueba: public.zip

Al descomprimir el archivo `public.zip` obtendréis la carpeta `public`, que contiene diez instancias y sus resultados. Sus nombres siguen el patrón:

`e5-<num>-<steps>.<ext>`

donde:

- `<num>`: El número de instancia, entre 1 y 10.
- `<steps>`: El número de pasos de la secuencia más corta que resuelve el puzle.
- `<ext>`: La extensión del archivo. Indica el contenido: 'i' si contiene una instancia u 'o' si contiene un resultado.

Es posible que algunas instancias tengan más de una solución válida, por lo que puede que algunas de vuestras soluciones no coincidan con las que contiene la carpeta. En cualquier caso, las soluciones deben coincidir en el número de pasos.

3.2. Archivo principal: e5.py

Contiene la estructura del programa. Debéis modificarlo teniendo en cuenta lo siguiente:

- Podéis añadir funciones o clases adicionales, pero no debéis modificar nada del programa principal. Modificar el programa principal supondrá un cero en la calificación del entregable. Tampoco podéis modificar la firma (el tipo) de las tres funciones que se utilizan en el programa principal.
- Podéis importar módulos de la biblioteca estándar de Python, pero la única biblioteca externa permitida es `algoritmia`. Por ejemplo, `NumPy` es una biblioteca externa y, por lo tanto, no está permitida.

Antes de continuar debemos señalar que, para detectar errores en el entorno de trabajo, el programa incluye una función, `_check_environment`, que se ejecuta automáticamente al lanzar el programa y comprueba que estén instaladas las versiones adecuadas de Python y de la biblioteca `algoritmia`.

Veamos la estructura de `e5.py` y las funciones que debéis implementar.

Por comodidad, el programa define estos dos tipos:

```
type Board = list[list[int]]    # El tablero como lista de listas
type Decision = str             # Un movimiento: 'D', 'U', 'R' o 'L'
```

El programa define el tipo `Data` para las instancias y el tipo `Result` para los resultados:

```
type Data = Board              # Una instancia es un tablero
type Result = str              # Una solución es una cadena de 'D', 'U', 'R' y 'L'.
```

Estos dos tipos se detallan más adelante, en la implementación de las funciones `read_data` y `process`.

El programa principal utiliza la estructura de tres funciones vista en clase:

```
data0 = read_data(sys.stdin)
result0 = process(data0)
show_result(result0)
```

La función `show_result` ya está implementada y no debéis modificarla. Debéis implementar las funciones `read_data` y `process`:

- `read_data(f: TextIO) -> Data:`
 - Entrada: Un archivo de texto con la instancia (`f` es un archivo, no un nombre de archivo).
 - Salida: Un tablero de 3×3 como una lista de listas de enteros.
- `process(data: Data) -> Result:`
 - Entrada: El tablero que devuelve `read_data`.
 - Salida: Una cadena de caracteres que representa la secuencia de movimientos más corta que resuelve el tablero. Utilizaremos los caracteres R (*Right*), L (*Left*), U (*Up*) y D (*Down*) para representar el movimiento del hueco.

Cómo ejecutar vuestro programa `e5.py`

Dado que el programa lee de la entrada estándar, se recomienda ejecutarlo desde el terminal que incorpora el propio PyCharm (el botón está en la parte inferior izquierda). Una vez abierto, utilizad el comando `cd` para ir al directorio donde están `e5.py` y `public`.

Una vez en el directorio, podéis ejecutar el programa con el siguiente comando:

- Linux y MacOS: `python3 e5.py < public/e5-01-4.i`
- Windows (CMD): `python e5.py < public\e5-01-4.i`
- Windows (PowerShell): `Get-Content public\e5-01-4.i | python e5.py`

4. Entrega en el aula virtual

La entrega consistirá en subir dos archivos a la tarea correspondiente del aula virtual, sólo debe subirlos uno de los miembros de la pareja. Estos son los dos archivos:

- `e5.py`: El archivo con el código del entregable.
- `alxxxxxx_alyyyyyy.txt`: Un archivo de texto que deberá contener el nombre de cada miembro de la pareja (un nombre por línea). Evidentemente, en el nombre del archivo tenéis que reemplazar `alxxxxxx` y `alyyyyyy` por los correspondientes a los dos miembros de la pareja. Si la entrega es individual, el archivo de texto deberá llamarse `alxxxxxx.txt`, reemplazando `alxxxxxx` por el que corresponda.

Vuestra entrega debe cumplir las siguientes restricciones:

- Sólo debéis subir los dos archivos solicitados, nada más.
- Los nombres de los dos archivos deben utilizar únicamente minúsculas.
- El carácter `'_'` que aparece en `alxxxxxx_alyyyyyy.txt` es el guion bajo, no utilizéis el carácter `'-'` ni ningún otro carácter similar como separador.

Información importante si utilizáis Windows para subir los archivos al aula virtual

Si usáis Windows, es probable que el administrador de archivos os oculte algunas extensiones de archivo, pues es la configuración por defecto de Windows. En ese caso, es posible que subáis algún archivo con doble extensión porque la extensión que ya tiene no la veréis y le añadiréis otra, (p. ej., `e5.py.py`). Como consecuencia, el nombre no será correcto y se penalizará la nota con un punto. Para evitar este error, configurad el administrador de archivos de Windows para que muestre las extensiones de todos los archivos.

5. Calificación del entregable

El entregable se calificará sobre 10 utilizando diez nuevas instancias. Las nuevas instancias se publicarán junto con las calificaciones para que podáis probarlas con vuestro programa.

Una instancia se considerará superada si se cumplen estas dos condiciones:

- La solución obtenida es óptima.
- El tiempo acumulado de ejecución de vuestras funciones `read_data` y `process` está por debajo del tiempo máximo de ejecución: un segundo.

Con las instancias más fáciles es posible obtener la solución óptima incluso con cotas poco ajustadas, con las intermedias se necesitan unas cotas más ajustadas. Para las más difíciles necesitaréis implementar una buena cota optimista y un state adecuado.

Importante: podéis detectar y corregir muchos errores en vuestro programa utilizando las instancias públicas, aunque superar las instancias públicas no garantiza superar las privadas, sobre todo si la implementación se ha “ajustado” específicamente para superar las públicas.

5.1. Medición de tiempos de ejecución

Para poder medir tiempos de forma consistente necesitamos utilizar un ordenador de referencia. El equipo elegido es `rei.dlsi.uji.es`. Las instancias se han generado para que cada una pueda resolverse en este equipo en menos de un segundo, con una implementación de state adecuada y una buena cota optimista.

Para obtener el tiempo de ejecución sobre el ordenador de referencia, podéis subir vuestro programa al sitio web:

<https://rei.dlsi.uji.es/algoritmia>

donde un servicio lo ejecutará para todas las instancias y mostrará el tiempo total de ejecución de vuestras funciones `read_data` y `process` para cada una de ellas.

Algunas consideraciones sobre el sitio web:

- Para conectaros desde fuera de la UJI, necesitaréis utilizar la VPN de la universidad. Si no sabéis cómo configurarla, podéis encontrar toda la información en

<https://connectaxarxa.uji.es/es/index.html#vpn>

Importante: Tened en cuenta que al activar la VPN recibiréis un correo de la UJI con un enlace que debéis pulsar para que la VPN no se cierre automáticamente a los 60 segundos (es el doble factor).

- Dado que el equipo no está conectado directamente a Internet, su certificado es autofirmado. Tendréis que aceptar una excepción de seguridad en el navegador para poder acceder al sitio web.
- El servicio sólo mide el tiempo de ejecución conjunto de vuestras funciones `read_data` y `process`, no comprueba que la solución que obtiene vuestro programa sea correcta.

5.2. Errores graves en el entregable

Se calificarán con un cero los entregables:

- Que no lean las instancias desde la entrada estándar, tal y como se indica en el apartado 2.1. Si no sabéis hacerlo, mirad el código que escribisteis en la primera sesión de prácticas. Si no lo entendéis, preguntad al profesor.
- Que hayan modificado el programa principal de `e5.py` o los tipos de cualquiera de las funciones que utiliza.
- Que no resuelvan el problema utilizando explícitamente el esquema de ramificación y acotación de la biblioteca `algoritmia` (`bab_scheme.py`).

También se penalizará que el contenido de la salida estándar no respete el formato que se especifica en el apartado 2.2. Una salida errónea puede tener dos causas:

- Una implementación de `show_result` que no respete el formato especificado. Se penalizará la nota con dos puntos. Dado que en este entregable se os proporciona la función `show_result` ya implementada, este error no debería suceder.
- Algún `print` de depuración olvidado en el código. Se penalizará la nota con un punto. Para que no os pase, podéis mostrarlos por la salida de error estándar, utilizando el parámetro `file=sys.stderr` de la función `print`:

```
print("Esto se muestra en la salida de error estándar", file=sys.stderr)
```

Por último, también se penalizará con un punto cada restricción incumplida en la entrega al aula virtual (ver el apartado 4). Revisad la entrega con detenimiento antes de subirla al aula virtual.

5.3. Penalización por copia

En caso de detectarse una copia, se aplicará la normativa de la universidad: la calificación de la evaluación continua (60 % de la nota final) será de cero en la primera convocatoria para todos los involucrados.